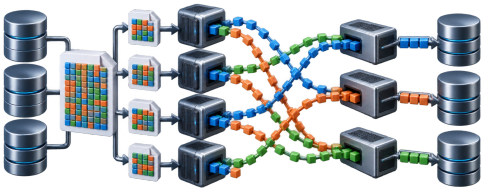
MapReduce : distribuer le calcul



BATCH PARALLÈLE

Du problème à la solution MapReduce

Pourquoi répartir le calcul ?

Le problème

- un seul serveur finit par saturer en **temps**, mémoire ou E/S ;
- beaucoup de traitements peuvent être découpés en sous-problèmes indépendants ;
- déplacer de gros volumes sur le réseau coûte cher ;
- les pannes deviennent normales lorsque le cluster grandit.

Le besoin n'est pas « plus de CPU », mais une façon structurée de répartir le calcul.

Principe : découper le traitement en tâches indépendantes, les exécuter en parallèle, puis regrouper leurs résultats.

déplacer le code

La localité des données évite les transferts coûteux

Option B : déplacer le code vers les données

- le bloc de **4 Go** est déjà présent sur N3 ;
- le programme ne pèse que **200 Ko** ;
- Hadoop essaie donc d'exécuter la tâche près du bloc ;
- on évite un transfert réseau inutile de plusieurs gigaoctets.

C'est le principe de data locality : déplacer le petit calcul plutôt que les grosses données.

Idée : garder le calcul près des données réserve le réseau aux échanges réellement nécessaires.

Cas réel : analyser les trajets des taxis de New York

Un même dataset, plusieurs questions métier Chaque trajet contient notamment :

- zone et heure de départ ;
- zone d'arrivée ;
- distance et montant total ;
- mode de paiement.

Questions possibles :

- quelles zones concentrent le plus de trajets ?
- quelles zones génèrent le plus de revenu ?
- à quelles heures la demande est-elle maximale ?

Le défi est d'obtenir ces indicateurs sans charger des dizaines de millions de lignes sur une seule machine.

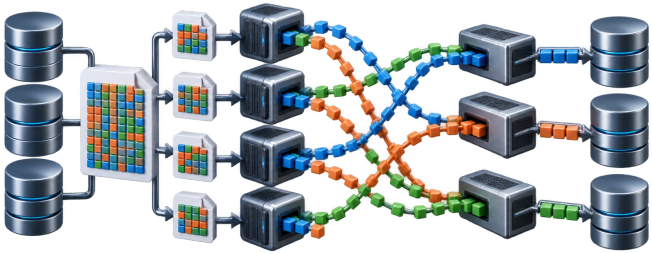
De la question métier au traitement distribué

Question métier Combien de trajets partent de chaque zone ?

1. lire les trajets ;
2. extraire la zone de départ ;
3. compter localement ;
4. regrouper les mêmes zones ;
5. agréger les comptes.

MapReduce transforme une question analytique en une suite de transformations parallèles et d'agrégations par clé.

Diviser, distribuer, agréger



- **1. Diviser** InputSplits indépendants
- **2. Distribuer** Maps parallèles + Shuffle
- **3. Agréger** Reduce par clé

Vous concevez Map et Reduce ; Hadoop automatise placement, échanges réseau, tri, relance et écriture.

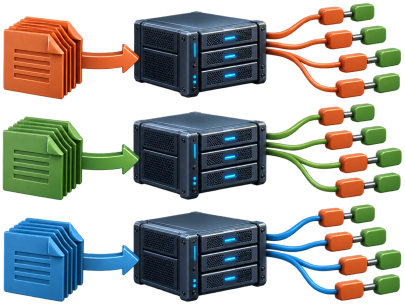
Tout passe par des paires (clé, valeur)

La clé définit l'axe d'agrégation

- la **clé** décide quels enregistrements seront regroupés ;
- la **valeur** transporte l'information à compter ou agréger ;
- deux questions différentes peuvent utiliser le même dataset avec des clés différentes.

```
(PULocationID, 1)
```

produit un comptage des trajets par zone de départ.



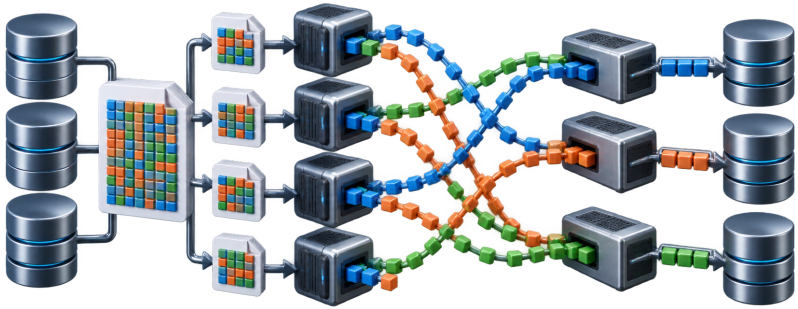
Une bonne conception MapReduce commence par une bonne clé.

La clé définit la question analytique

clé = zone	nombre de trajets par zone
clé = heure	activité par heure
clé = jour	activité par jour
clé = payment_type	répartition des usages par paiement
clé = produit	chiffre d'affaires par produit

Le choix de la clé n'est pas un détail de programmation : il traduit la question métier.

Pipeline MapReduce complet



HDFS InputSplits Map Shuffle/Sort Reduce

HDFS stocke ; les splits alimentent les Maps ; le Shuffle regroupe par clé ; Reduce produit le résultat final.

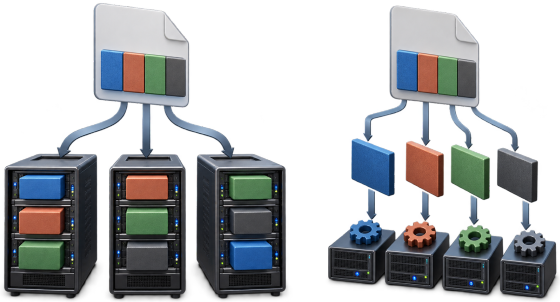
HDFS Block et InputSplit : ne pas confondre

HDFS Block = stockage

- unité logique de stockage HDFS ;
- répliquée pour tolérer les pannes ;
- taille configurable.

InputSplit = travail

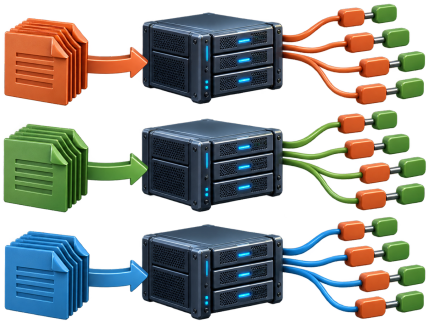
- unité logique de traitement ;
- alimente généralement une tâche Map ;
- dépend de l'InputFormat et du caractère splittable.



Le bloc répond à « où les données sont stockées ? » ; le split répond à « quel morceau de travail confier au Mapper ? ».

Phase Map : transformation parallèle

- chaque Mapper traite son fragment indépendamment ;
- il transforme les enregistrements un par un ;
- il émet des couples `(clé, valeur)` ;
- plusieurs Mappers s'exécutent en parallèle.



```
trip -> (PULocationID, 1)
```

Mini-activité : choisir la bonne clé

Objectif

Compter le nombre de trajets par zone de départ.

Trajet	PULocationID	Distance	Total
T1	132	17,4	52,50
T2	161	2,8	16,80
T3	132	9,2	31,20
T4	237	1,9	12,40

Q1 Quelle clé doit produire le Mapper ?

Q2 Quelle valeur minimale suffit pour un comptage ?

À vous : formulez d'abord la paire (clé, valeur).

Mini-activité : choisir la bonne clé

Objectif

Compter le nombre de trajets par zone de départ.

Trajet	PULocationID	Distance	Total
T1	132	17,4	52,50
T2	161	2,8	16,80
T3	132	9,2	31,20
T4	237	1,9	12,40

Q1 Quelle clé doit produire le Mapper ?

Correction : `PULocationID`

Q2 Quelle valeur minimale suffit pour un comptage ?

Mini-activité : choisir la bonne clé

Objectif

Compter le nombre de trajets par zone de départ.

Trajet	PULocationID	Distance	Total
T1	132	17,4	52,50
T2	161	2,8	16,80
T3	132	9,2	31,20
T4	237	1,9	12,40

Q1 Quelle clé doit produire le Mapper ?

Correction : `PULocationID`

Q2 Quelle valeur minimale suffit pour un comptage ?

Correction : `1`

Mini-activité : choisir la bonne clé

Objectif

Compter le nombre de trajets par zone de départ.

Trajet	PULocationID	Distance	Total
T1	132	17,4	52,50
T2	161	2,8	16,80
T3	132	9,2	31,20
T4	237	1,9	12,40

Q1 Quelle clé doit produire le Mapper ?

Correction : `PULocationID`

Q2 Quelle valeur minimale suffit pour un comptage ?

Correction : `1`

Couples émis : `(132,1), (161,1), (132,1), (237,1)`

La clé détermine le regroupement ; la valeur porte la mesure à agréger.

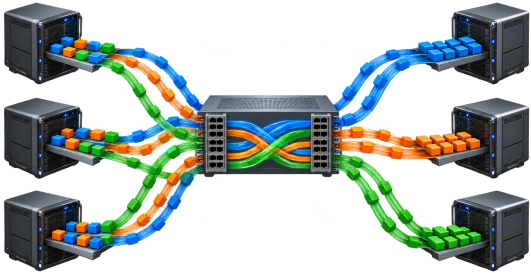
Shuffle + Sort : la phase qui fait bouger les données

1. Partition
choisit le Reducer d'une clé.

2. Shuffle
transfère les sorties Map sur le réseau.

3. Sort / Group
trie les clés et rassemble leurs valeurs.

Cette phase implique souvent **réseau + sérialisation + E/S disque + tri**.



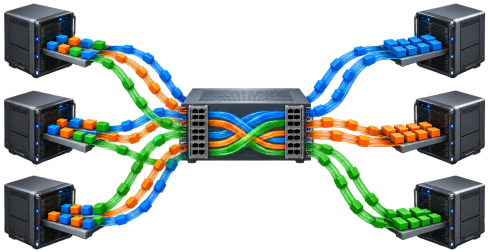
Le Shuffle est souvent l'un des coûts dominants d'un job MapReduce.

Le Shuffle en chiffres : pourquoi optimiser ?

Hypothèse simple

- 100 millions d'événements;
- 20 octets de sortie Map par événement;
- avant compression et overhead réseau.

```
100 000 000 x 20 octets ≈ 2 Go
```



Réduire les sorties Map avant le réseau peut changer radicalement le coût du job.

Mini-activité : que reçoit le Reducer ?

Sorties produites par trois Mappers

Mapper 1	Mapper 2	Mapper 3
(A,1)	(A,1)	(B,1)
(B,1)	(C,1)	(C,1)
(C,1)	(A,1)	(C,1)

Question

Après Shuffle + Sort, que recevra le Reducer chargé de la clé A ? Pourquoi certaines valeurs doivent-elles traverser le réseau avant d'être regroupées ?

À vous : suivez les couples de clé A produits par les trois Mappers.

Mini-activité : que reçoit le Reducer ?

Sorties produites par trois Mappers

Mapper 1	Mapper 2	Mapper 3
(A,1)	(A,1)	(B,1)
(B,1)	(C,1)	(C,1)
(C,1)	(A,1)	(C,1)

Question

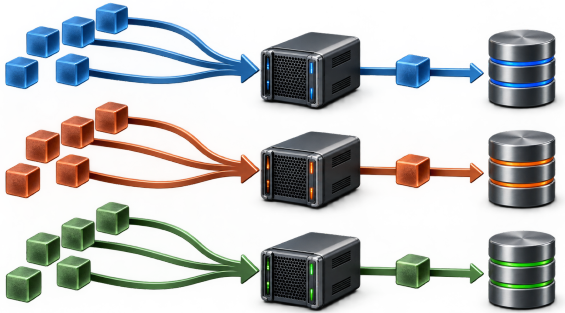
Après Shuffle + Sort, que recevra le Reducer chargé de la clé A ? Pourquoi certaines valeurs doivent-elles traverser le réseau avant d'être regroupées ?

Correction : le Reducer de A reçoit `A -> [1,1,1]` : un couple vient du Mapper 1 et deux du Mapper 2. Le Shuffle achemine ces couples vers le Reducer associé à leur clé, parfois depuis un autre nœud via le réseau.

Phase Reduce : agréger par clé

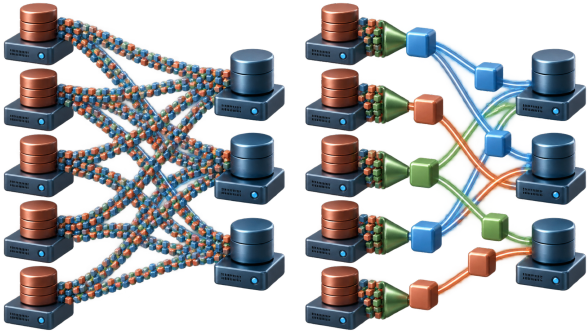
- un Reducer reçoit une clé et toutes ses valeurs ;
- il applique une agrégation ;
- il produit un résultat final par clé ;
- la sortie est généralement écrite dans HDFS.

```
Zone 132 -> [12, 7, 5, 9] -> 33
```



Combiner : réduire le trafic avant le Shuffle

- pré-agrégation locale près du Mapper ;
- peut être appelé **zéro, une ou plusieurs fois** ;
- ne doit jamais changer la sémantique du résultat ;
- adapté à somme, comptage, min, max ;
- pas automatiquement sûr pour toute opération.



Un Combiner n'est pas « un petit Reducer » : sa correction doit rester vraie quelle que soit sa fréquence d'exécution.

Combiner sur les taxis : de 4 messages à 1

Sans pré-agrégation

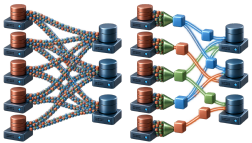
```
(132,1)
(132,1)
(132,1)
(132,1)
```

Quatre couples doivent être transférés.

Avec Combiner

```
(132,4)
```

Une seule valeur part sur le réseau.



Plus une clé se répète localement, plus le Combiner peut réduire le trafic.

Moyenne distribuée : pourquoi `mean(mean(...))` est faux ?

- **Machine A** 10 trajets, distance moyenne = 4 km
- **Machine B** 100 trajets, distance moyenne = 10 km

```
(4 + 10) / 2 = 7 km <- faux
```

- **Représentation correcte** Conserver localement :

```
(sum_distance, count)
```

Puis agréger :

```
sum(sum_distance) / sum(count)
```

En distribué, il faut préserver suffisamment d'information pour recomposer exactement le résultat global.

Data locality : exécuter le calcul près des données

- Hadoop essaie d'exécuter les Maps sur les nœuds qui possèdent les blocs ;
- sinon, il privilégie un nœud proche dans la topologie ;
- les sorties Map restent locales jusqu'au Shuffle.

```
 bloc = 4 Go
programme = 200 Ko
```

Déplacer quelques Ko de code coûte souvent beaucoup moins que déplacer plusieurs Go de données.

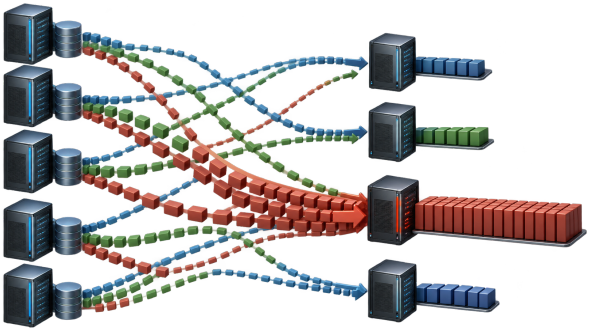
Data skew : quand une clé concentre trop de données

Exemple mobilité

- Zone A : 65 % des trajets ;
- Zone B : 10 % ;
- Zone C : 8 % ;
- le reste est dispersé.

Conséquence

- un Reducer reçoit une charge disproportionnée ;
- les autres terminent tôt ;
- le job attend le **straggler**.



Salting : répartir temporairement une clé chaude

Principe Une clé dominante est découpée temporairement :

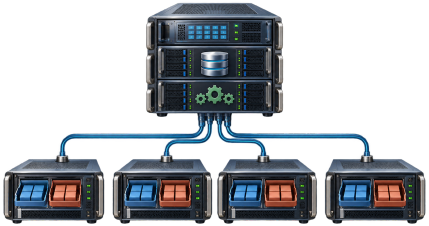
```
Airport -> Airport_1, Airport_2,
Airport_3, Airport_4
```

Puis une seconde agrégation rassemble le résultat final.

- meilleure répartition de charge ;
- une étape supplémentaire ;
- stratégie dépendante de l'opération.

MRv1 : architecture historique

- un **JobTracker** central reçoit et planifie les jobs ;
- des **TaskTrackers** exécutent les tâches ;
- les ressources sont organisées en slots Map et Reduce ;
- une tâche échouée peut être relancée ailleurs.



Le point faible : un composant central concentre trop de responsabilités (gestion des ressources, planification, suivi des tâches, gestion des erreurs).

Pourquoi MRv1 a évolué ?

MRv1

- JobTracker = ressources + ordonnancement + suivi ;
- slots Map/Reduce rigides ;
- cluster fortement centré sur MapReduce.

YARN

- gestion globale des ressources séparée ;
- gestion propre à chaque application ;
- allocations dynamiques ;
- plusieurs moteurs possibles.

La scalabilité vient aussi de la séparation des rôles, pas seulement de l'ajout de machines.

YARN : gérer les ressources du cluster

- **ResourceManager** arbitre global des ressources.
- **NodeManager** agent de chaque machine ; lance et surveille les allocations.
- **ApplicationMaster** propre à l'application ; négocie les ressources et suit les tâches.

YARN sépare la gestion des ressources de la logique du moteur de traitement.

YARN : exécution d'un job MapReduce

1. le client soumet l'application au ResourceManager ;
2. YARN démarre un ApplicationMaster ;
3. l'ApplicationMaster négocie des containers ;
4. les NodeManagers lancent Map puis Reduce ;
5. l'ApplicationMaster suit la progression.

YARN ne remplace pas MapReduce : il organise les ressources utilisées par MapReduce et d'autres moteurs.

YARN container ≠ Docker container

YARN container Allocation de ressources sur **un nœud du cluster** :

- mémoire ;
- CPU / vcores ;
- ressources nécessaires à l'exécution d'un processus.

Un même job peut utiliser **plusieurs containers répartis sur plusieurs nœuds**.

Docker container Environnement d'exécution **isolé** contenant :

- l'application ;
- ses dépendances ;
- son système de fichiers ;
- les mécanismes d'isolation.

WordCount : comprendre MapReduce avec un exemple simple

1. Entrée Chaque ligne du fichier est lue par un Mapper.

```
 jour leve notre grisaille
trottoir notre ruelle notre tour
jour leve notre envie
```

2. Map Pour chaque mot rencontré, le Mapper émet :

```
(mot, 1)
```

Exemple :

```
(jour,1) , (notre,1), (notre,1)
```

3. Shuffle + Sort Toutes les valeurs associées au même mot sont regroupées.

```
 jour -> [1,1]
notre -> [1,1,1,1]
```

4. Reduce Le Reducer additionne les valeurs de chaque mot.

```
 jour -> 2
notre -> 4
```

WordCount illustre le principe de MapReduce : Map produit des paires clé–valeur, Shuffle regroupe les mêmes clés et Reduce agrège leurs valeurs.

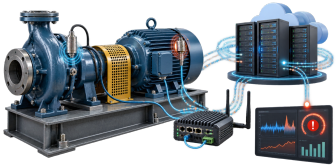
Du WordCount à de vrais problèmes

Logs web clé = URL

nombre de requêtes par URL

E-commerce clé = produit

chiffre d'affaires par produit



IoT industriel clé = machine

nombre d'alertes par équipement

Même structure abstraite, autre clé = autre problème distribué.

Cas NYC Taxi : compter les trajets par zone

Map

```
trip -> (PULocationID, 1)
```

Shuffle + Sort

```
132 -> [1,1,1,...]
```

Reduce

```
132 -> 15482 trajets
```

La clé `PULocationID` transforme des millions de trajets en un indicateur de demande par zone.

Du résultat brut à une décision

Sortie brute

```
 132 -> 15482
161 -> 12120
237 -> 10940
...
```

Après enrichissement

- associer l'identifiant à une vraie zone ;
- classer les zones ;
- visualiser une carte de demande ;
- guider une décision opérationnelle.

MapReduce produit d'abord une agrégation ; sa valeur apparaît quand elle est replacée dans le contexte métier.

Changer la clé = changer la question analytique

Question	Clé Map	Interprétation
Nombre de trajets par zone	`PULocationID`	demande géographique
Activité par heure	`heure(pickup)`	profil temporel
Usage par mode de paiement	`payment_type`	comportement de paiement
Revenu par zone	`PULocationID`	montant agrégé par zone
Distance moyenne par zone	`PULocationID`	mobilité / typologie de trajet

La clé encode la question posée aux données ; la valeur encode la mesure à agréger.

Agrégation avancée : moyenne distribuée correcte

Map

```
(zone, (distance, 1))
```

Combiner possible

```
(zone, (sum_distance, count))
```

Reduce

```
mean = sum(sum_distance) / sum(count)
```

Pourquoi ? Parce que les partitions n'ont pas forcément la même taille.

En distribué, une statistique correcte dépend autant de la représentation intermédiaire que de la formule finale.

Mini-activité : concevoir un traitement distribué

Objectif

Trouver le Top-10 des zones générant le plus de revenu entre 7h et 10h.

1. Quel filtre appliquer avant l'émission des couples ?
2. Quelle clé utiliser pour calculer le revenu par zone ?
3. Quelle valeur doit transporter le Mapper ?
4. Un Combiner est-il possible ? Pourquoi ?
5. Une seule étape MapReduce suffit-elle pour un Top-10 global ?

Partez de la question métier, puis définissez clé, valeur, agrégation et coût du Shuffle.

Correction : concevoir un traitement distribué

Top-10 des zones par revenu entre 7h et 10h

Map : filtrer et émettre

- conserver les trajets dont l'heure de départ est dans `[07:00, 10:00)`;
- émettre `(PULocationID, total_amount)`;
- ici, `total_amount` définit le revenu comme montant total facturé.

Combiner : additionner localement

Somme partielle des montants pour chaque zone, avant le Shuffle.

Reduce : calculer le revenu

```
zone -> somme(total_amount)
```

Top-10 : classer globalement

Un premier job calcule le total par zone. Un second sélectionne les dix plus grands totaux; les Top-10 locaux des Reducers ne suffisent pas à garantir le Top-10 global.

Le Combiner est correct ici car la somme est associative; la définition du revenu doit rester explicite.

Limites de MapReduce : pourquoi Spark ensuite ?

Très adapté

- batch massif ;
- scans séquentiels ;
- agrégations par clé ;
- ETL ;
- traitements tolérants aux pannes.

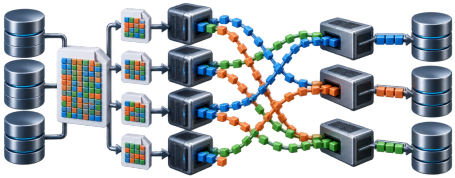
Moins adapté

- Shuffle très volumineux ;
- nombreuses étapes matérialisées sur disque ;
- calculs itératifs ;
- faible latence interactive ;
- pipelines multi-étapes complexes.

Spark ne rend pas MapReduce « faux » : il optimise surtout les scénarios où les intermédiaires disque deviennent pénalisants.

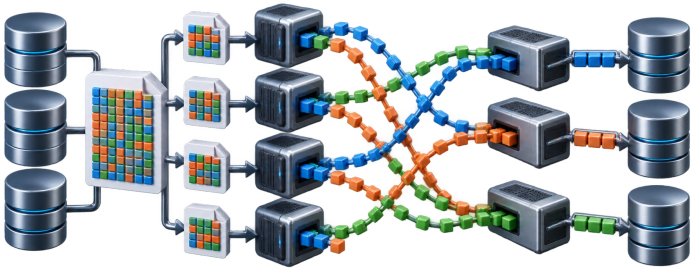
Comment penser un problème MapReduce ?

1. **Que veux-je calculer ?**
2. **Quelle clé représente ma question ?**
3. **Que doit émettre Map ?**
4. **Combien de données vont traverser Shuffle ?**
5. **Que doit agréger Reduce ?**



Concevoir MapReduce, c'est d'abord raisonner sur la donnée et sur le coût du mouvement des données.

MapReduce : la carte mentale à reconstruire



Data locality **Combiner** **Shuffle = réseau** **Data skew** **YARN = ressources**

HDFS stocke et répartit ; Map transforme localement ; Shuffle/Sort regroupe par clé ; Reduce agrège ; YARN alloue les ressources.

Évaluation de compétences

HDFS + MAPREDUCE

14 questions pour vérifier les notions du chapitre

Accéder au site d'évaluation

Scannez le QR code ou ouvrez le lien

Scan me!

Site web d'évaluation des compétences.

`takwabenaichagader.github.io/mapreduce-part2-training/`

Le QCM commence à la diapositive suivante.

Question 1

Parmi les propositions suivantes, laquelle décrit le mieux HDFS ?

A. Une base relationnelle transactionnelle à faible latence.

B. Un système de fichiers distribué optimisé pour de grands volumes et un fort débit.

C. Un ordonnanceur de conteneurs Docker.

D. Un langage de programmation pour le streaming temps réel.

Question 1 : réponse

Réponse B HDFS est un système de fichiers distribué conçu pour de grands jeux de données, la tolérance aux pannes et le **débit**, plutôt que la faible latence interactive.

Question 2

Où passent les données lors de la lecture d'un fichier HDFS ?

A. DataNode → NameNode → client.

B. DataNode → client directement, après consultation des métadonnées du NameNode.

C. NameNode uniquement : il stocke les blocs et les métadonnées.

D. ZooKeeper transporte les blocs vers le client.

Question 2 : réponse

Réponse B Le NameNode est sur le chemin des **métadonnées**, pas sur le chemin principal des données utilisateur. Après localisation, le client lit directement auprès des DataNodes.

Question 3

Quel composant remplace automatiquement le NameNode actif dans une architecture HDFS HA ?

A. Secondary NameNode.

B. Standby NameNode.

C. DataNode principal.

D. Reducer.

Question 3 : réponse

Réponse B — Standby NameNode Le Secondary NameNode produit historiquement des checkpoints ; il n'est pas le serveur de failover automatique.

Question 4

Un fichier possède un facteur de réplication de 3. Que signifie ce nombre ?

A. Le fichier est découpé en exactement 3 blocs.

B. Chaque bloc doit normalement disposer de 3 réplicas.

C. Trois Reducers seront lancés.

D. Trois NameNodes sont nécessaires.

Question 4 : réponse

Réponse B Chaque bloc vise normalement trois réplicas lorsque le facteur de réplication vaut 3.

Question 5

Quel énoncé distingue correctement HDFS Block et InputSplit ?

A. Ils désignent toujours strictement le même objet.

B. Le Block concerne le stockage ; l'InputSplit concerne le travail MapReduce.

C. L'InputSplit est une copie physique du bloc.

D. Le Block est créé par le Reducer.

Question 5 : réponse

Réponse B Le bloc est une notion HDFS ; le split est une notion de calcul MapReduce. Ils peuvent être proches en taille, mais ils ne doivent pas être confondus.

Question 6

Quelle phase est responsable du regroupement des valeurs ayant la même clé ?

A. Split.

B. Map uniquement.

C. Shuffle + Sort.

D. Réplication HDFS.

Question 6 : réponse

Réponse C Shuffle + Sort regroupe les valeurs partageant la même clé avant qu'elles soient présentées au Reducer.

Question 7

À quoi sert principalement un Combiner ?

A. Répliquer les blocs HDFS.

B. Pré-agréger localement les sorties Map pour réduire le trafic réseau.

C. Remplacer le Reducer dans tous les cas.

D. Choisir le NameNode actif.

Question 7 : réponse

Réponse B Une pré-agrégation locale peut diminuer fortement le trafic réseau, à condition que l'opération reste correcte lorsqu'elle est appliquée localement puis globalement.

Question 8

Pourquoi Hadoop cherche-t-il à rapprocher le calcul des données ?

A. Pour économiser la bande passante réseau et augmenter le débit.

B. Pour éviter l'utilisation du CPU.

C. Pour supprimer la réplication HDFS.

D. Pour transformer HDFS en base relationnelle.

Question 8 : réponse

Réponse A — data locality Déplacer le calcul vers les données économise de la bande passante et améliore le débit global du cluster.

Question 9

Dans YARN, quel composant est l'agent de chaque machine ?

A. NodeManager.

B. NameNode.

C. JobTracker.

D. Secondary NameNode.

Question 9 : réponse

Réponse A Le **NodeManager** est l'agent de chaque machine dans YARN ; il lance et surveille les allocations locales.

Question 10

Un YARN container est-il un Docker container ?

A. Oui, les deux termes sont strictement synonymes.

B. Non. YARN alloue des ressources ; Docker fournit un environnement d'exécution isolé.

Question 10 : réponse

Réponse B YARN alloue des ressources ; Docker fournit un environnement d'exécution isolé. Les deux concepts sont distincts, même si YARN peut utiliser Docker comme runtime.

Question 11 : combien de Mappers ?

Deux fichiers HDFS : **513 MiB** et **515 MiB**. Taille de bloc/split supposée : **512 MiB**. Combien de Maps avec les hypothèses par défaut ?

Données

Fichier 1 : 513 MiB
Fichier 2 : 515 MiB
Cluster : au moins 5 Maps peuvent s'exécuter simultanément.

Combien de Maps le traitement crée-t-il ?

Question 11 : réponse

4 Mappers

- 513 MiB $\rightarrow$ 2 blocs / splits ;
- 515 MiB $\rightarrow$ 2 blocs / splits ;
- total $\rightarrow$ 4 Maps dans cette hypothèse.

La capacité de 5 Maps en parallèle ne crée pas une cinquième tâche : elle indique seulement combien peuvent s'exécuter simultanément.

Question 12 : taille et nombre de Maps

Un fichier fait **5000 MiB**. Avec une taille de split proche de **512 MiB**, combien de Maps obtient-on approximativement ?

A. 1

B. 5

C. 10

D. 512

Question 12 : réponse

Réponse C — 10 Maps $\lceil 5000/512 \rceil = 10$. Cette réponse suppose un fichier splittable et un InputFormat dont la taille de split suit approximativement la taille de bloc.

Question 13 : combien de copies de blocs ?

`logs1.txt` = 1036 MiB, `logs2.txt` = 500 MiB, blocs de **512 MiB**, facteur de réplication = **4**. Combien de réplicas au total ?

A. 4

B. 12

C. 16

D. 20

Question 13 : réponse

- `logs1` : $\lceil 1036/512 \rceil = 3$ blocs $\times 4 = 12$ réplicas.
- `logs2` : $\lceil 500/512 \rceil = 1$ bloc $\times 4 = 4$ réplicas.

Total : 12 + 4 = 16 réplicas de blocs — réponse C.

Question 14 : WordCount

Pour compter les occurrences des mots, quel couple doit émettre la fonction Map ?

A. (document, mot)

B. (mot, 1)

C. (1, mot)

D. (mot, document entier)

Question 14 : réponse

Réponse B — `(mot, 1)`

```
def mapper(text):
   for word in text.split():
      emit(word, 1)

def reducer(word, counts):
   emit(word, sum(counts))
```

À retenir avant le TP

- HDFS découpe et distribue les fichiers, puis réplique les blocs pour résister aux pannes.
- Le NameNode gère les métadonnées ; les DataNodes portent les données.
- MapReduce transforme un gros calcul en tâches Map, Shuffle/Sort et Reduce.
- La data locality vise à réduire le trafic réseau.
- YARN sépare la gestion des ressources de la logique du moteur de traitement.

Ces principes expliquent une grande partie des systèmes modernes de Data Engineering.

Références techniques

- Apache Hadoop — HDFS Architecture.
- Apache Hadoop — HDFS High Availability with QJM.
- Apache Hadoop — MapReduce Tutorial.
- Apache Hadoop — YARN Architecture.
- Apache Hadoop — FileSystem Shell.

Support révisé pour l'année universitaire 2026/2027.